

Post Quantum Proofs of Reed-Solomon Codes with LeanAIR

Status: research writeup **Code:** [crates/lean-air](#) **Implementation:** [frisitano/leanMultisig@232308f2](#) **Author:** Tau Lepton

TL;DR

LeanAIR evaluates a direct prover for a post-quantum Reed-Solomon data-availability commitment. On a 192-vCPU Graviton4 instance, the LeanAIR single-proof measurement is about **7.5 MiB/s**, compared with the available single-proof `lean-da` measurements of **1.37 MiB/s** for the LeanVM baseline parity check and **1.19 MiB/s** for the LeanVM column-commit parity check. LeanAIR also reaches about **25 MiB/s aggregate proving throughput** using parallel batching, but parallelized `lean-da` proof generation was not measured here. The prover is organized around the commitment-level operations:

1. hash each fixed-size codeword cell;
2. chain systematic cell digests into row commitments;
3. Merkle-commit the cell columns;
4. bind row and column roots into one public commitment;
5. prove each row is a Reed-Solomon codeword using the barycentric parity identity.

ZKVM Circuit

Cell digest grid

Each codeword row is split into fixed cells. One cell is n extension-field elements from that row.

$$\text{cell}_{i,j} = C_i[jn \dots (j + 1)n] \in \mathbb{E}^n$$

$$q_{i,j} = H_{\text{chain}}(\text{chunks}(\text{limbs}_{\mathbb{F}}(\text{cell}_{i,j}))) \in \mathbb{F}^8$$

systematic cells				parity cells			
row	cell 0	cell 1	cell 2	cell 3	cell 4	cell 5	row commitments
codeword1	q[0,0]	q[0,1]	q[0,2]	q[0,3]	q[0,4]	q[0,5]	rowCom_0
codeword2	q[1,0]	q[1,1]	q[1,2]	q[1,3]	q[1,4]	q[1,5]	rowCom_1
codeword3	q[2,0]	q[2,1]	q[2,2]	q[2,3]	q[2,4]	q[2,5]	rowCom_2
codeword4	q[3,0]	q[3,1]	q[3,2]	q[3,3]	q[3,4]	q[3,5]	rowCom_3
column commitments	col_0	col_1	col_2	col_3	col_4	col_5	

Commitment definitions

$$\text{rowCommit}_i = H_{\text{chain}}(q_{i,0}, q_{i,1}, q_{i,2})$$

$$R_{\text{rows}} = H_{\text{chain}}(\text{rowCommit}_0, \text{rowCommit}_1, \dots)$$

$$\text{col}_j = \text{Merkle}(q_{0,j}, q_{1,j}, \dots)$$

$$R_{\text{col}} = \text{Merkle}(\text{col}_0, \text{col}_1, \text{col}_2, \dots)$$

$$D = H(R_{\text{rows}}, R_{\text{col}})$$

Row commitments

chain only systematic cell digests
 $\text{rowCommit}_i \Rightarrow R_{\text{rows}}$

Column commitments

Merkle every cell-column digest
 $\text{col}_j \Rightarrow R_{\text{col}}$

Cell opening

hash one cell, then verify its path
in the column tree

$$q_{i,j} \xrightarrow{\text{path}} \text{col}_j \rightarrow R_{\text{col}} \rightarrow D$$

Column opening

recompute column root
no intermediate column-tree nodes

$$(q_{0,j}, \dots, q_{B-1,j}) \mapsto \text{col}_j \rightarrow R_{\text{col}} \rightarrow D$$

The resulting proof uses a dedicated Poseidon table, WHIR openings of that table, and two families of virtual linear consistency claims. The row low-degree test is expressed as a sparse linear claim over the same committed trace columns that feed the cell hashes, so the values checked for Reed-Solomon validity are exactly the values committed by the cell and column commitments.

1. The Experiment

The earlier implementation used a LeanVM program:

- it witnesses row codewords;
- calls Poseidon precompiles for cell hashes and commitments;
- computes barycentric row checks inside VM execution;
- proves the whole VM trace.

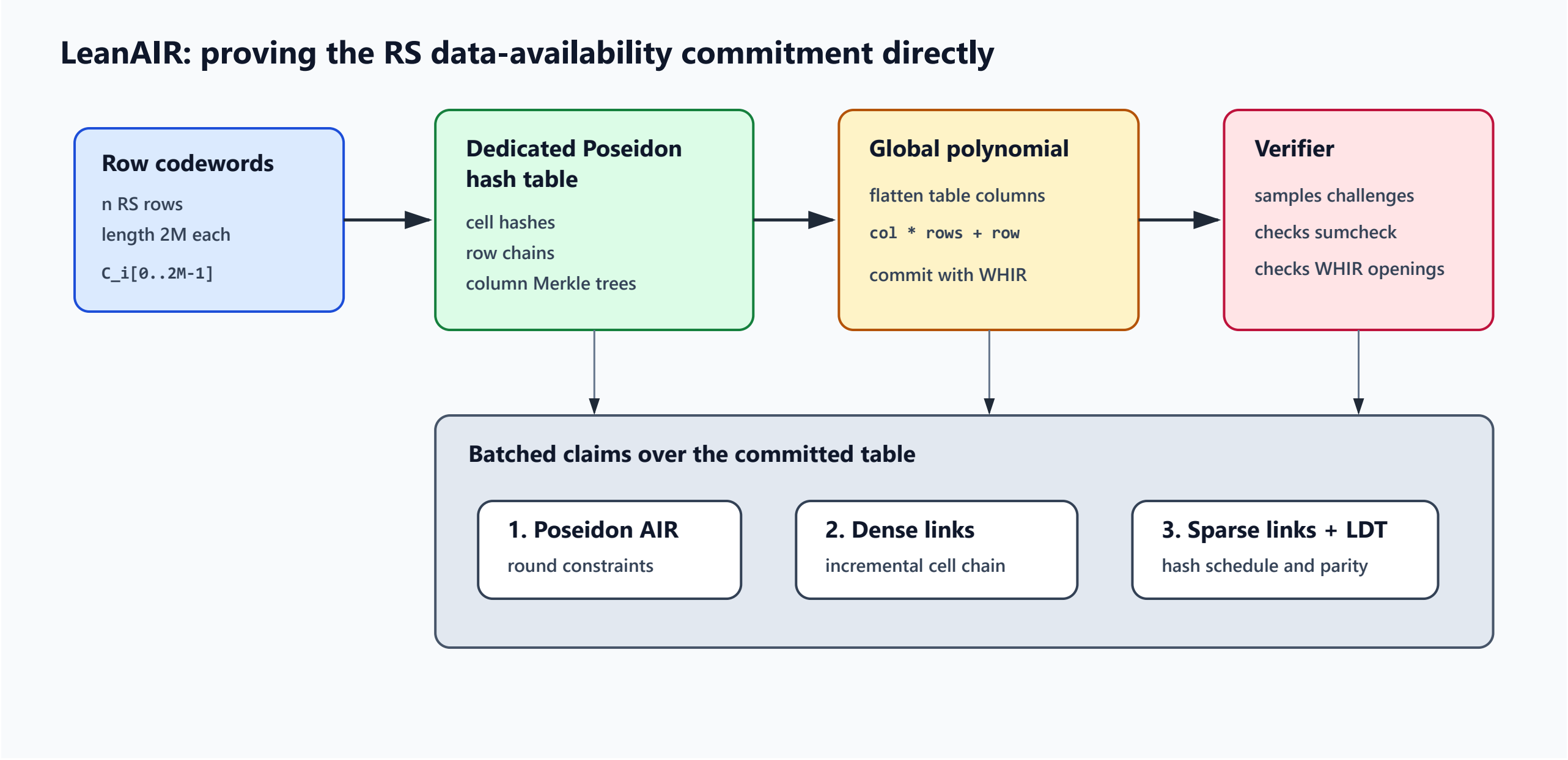
In that implementation, the proof statement includes general VM execution components in addition to the commitment-specific computation:

- instruction decoding;
- bytecode accumulation;
- memory indirection;
- generic precompile request routing;

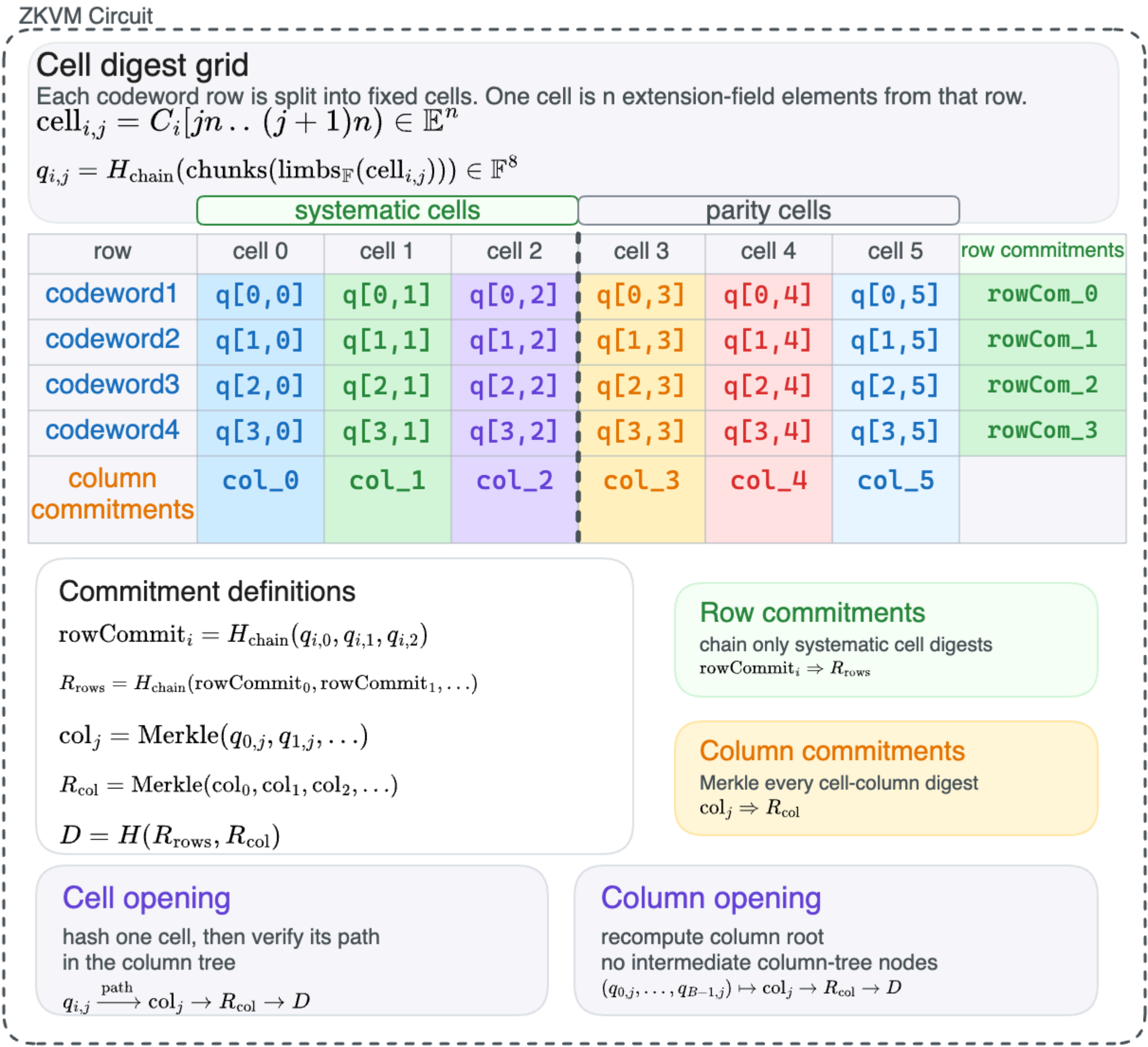
- repeated extension arithmetic through the VM execution model.

LeanAIR keeps the same proof backend family, field stack, Poseidon permutation, WHIR PCS, and Fiat-Shamir transcript shape, but removes the VM layer. The witness is lowered directly into a dedicated table whose rows are Poseidon16 compressions and whose columns are the Poseidon round witnesses.

The design represents the cryptographic and coding-theoretic work directly in the proof trace. Once the table layout is public, the proof can be organized around three ingredients: local Poseidon constraints, global linear wiring constraints, and a Reed-Solomon row check.

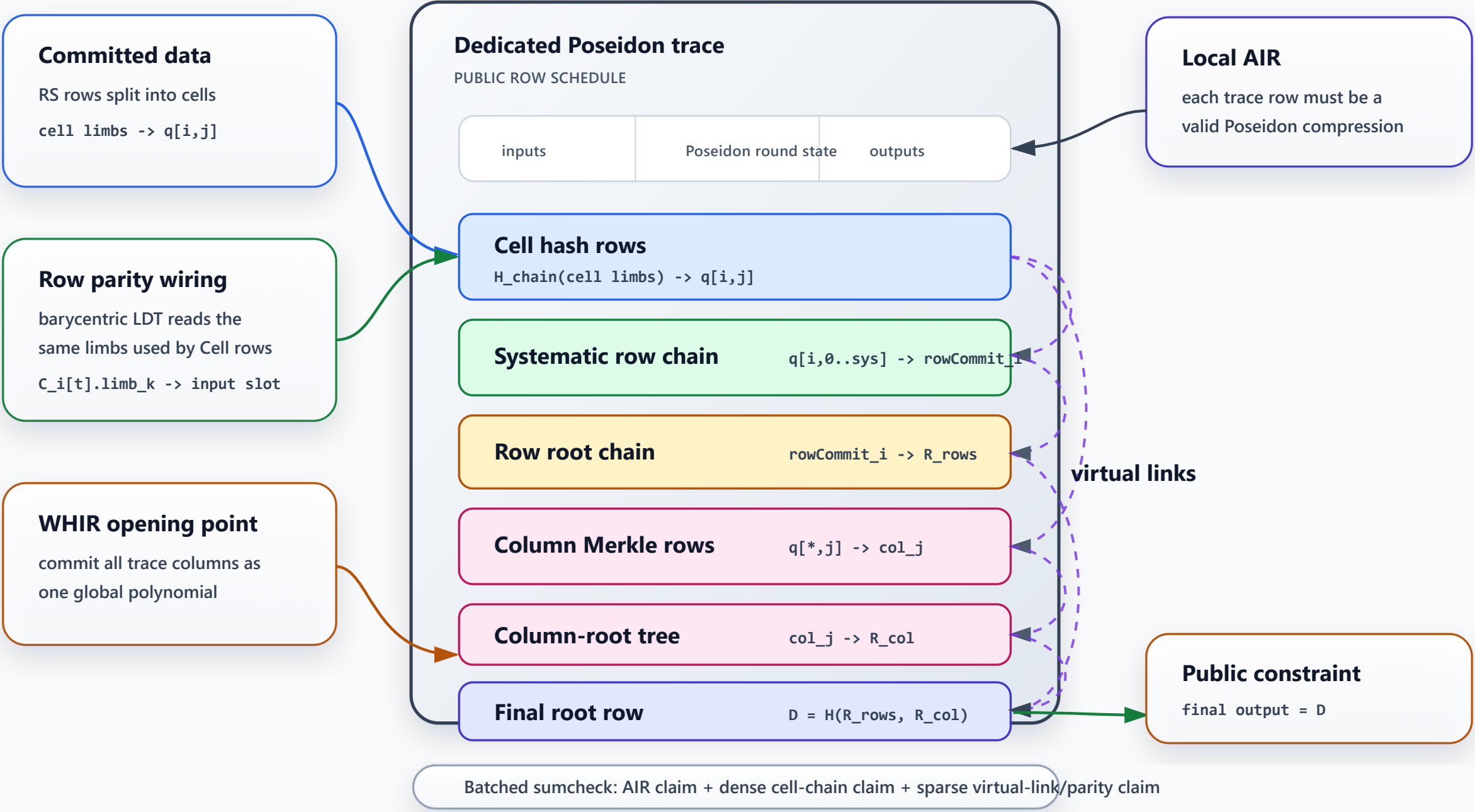


2. Commitment Shape



LeanAIR Poseidon trace chip

One deterministic hash trace; local AIR checks rows, virtual claims wire the public commitment and row-code test.



The committed table columns are flattened into one global multilinear polynomial:

```
global[column * padded_rows + row] = trace[column][row]
```

WHIR commits to this polynomial. The verifier samples a row-domain point, receives all column evaluations at that point, and checks one batched claim:

- 1. local Poseidon AIR constraints;
- 2. dense cell-chain links;
- 3. sparse virtual linear constraints for schedule wiring and row parity.

```
claim = AIR_claim + eta * cell_link_claim + eta^2 * sparse_linear_claim
```

The dense links cover incremental cell hashes:

```
output(prev_chunk)[limb] = input(next_chunk)[limb]
```

The sparse links cover the public hash schedule:

- systematic row digests read cell hash outputs;
- row-root rows read row digest outputs;
- column Merkle parents read child digest outputs;
- the column-root tree reads column roots;
- the final root reads `R_rows` and `R_col`.

The row low-degree test is another sparse linear identity over the same committed columns. For each row codeword `C_i`, let `w` be a primitive 2^M -th root of unity and `u` = `w`². The even and odd halves define:

```
L_i(X): interpolates {(u^j, C_i[2j])}
R_i(X): interpolates {(w * u^j, C_i[2j+1])}
```

A valid row satisfies `L_i(X) = R_i(X)`. At a verifier challenge `r`, the barycentric form is:

```
sum_j slice_L[j](r) * C_i[2j] - sum_j slice_R[j](r) * C_i[2j+1] = 0
```

with:

```
slice_L[j](r) = (r^M - 1) / (r * u^{-j} - 1)
slice_R[j](r) = -(r^M + 1) / (r * w^{-1} * u^{-j} - 1)
```

LeanAIR aggregates all rows using a challenge derived from the public commitment:

```
alpha = Poseidon16("LEANAIR row alpha" || log_m || n_rows || cell_len_ext || D)

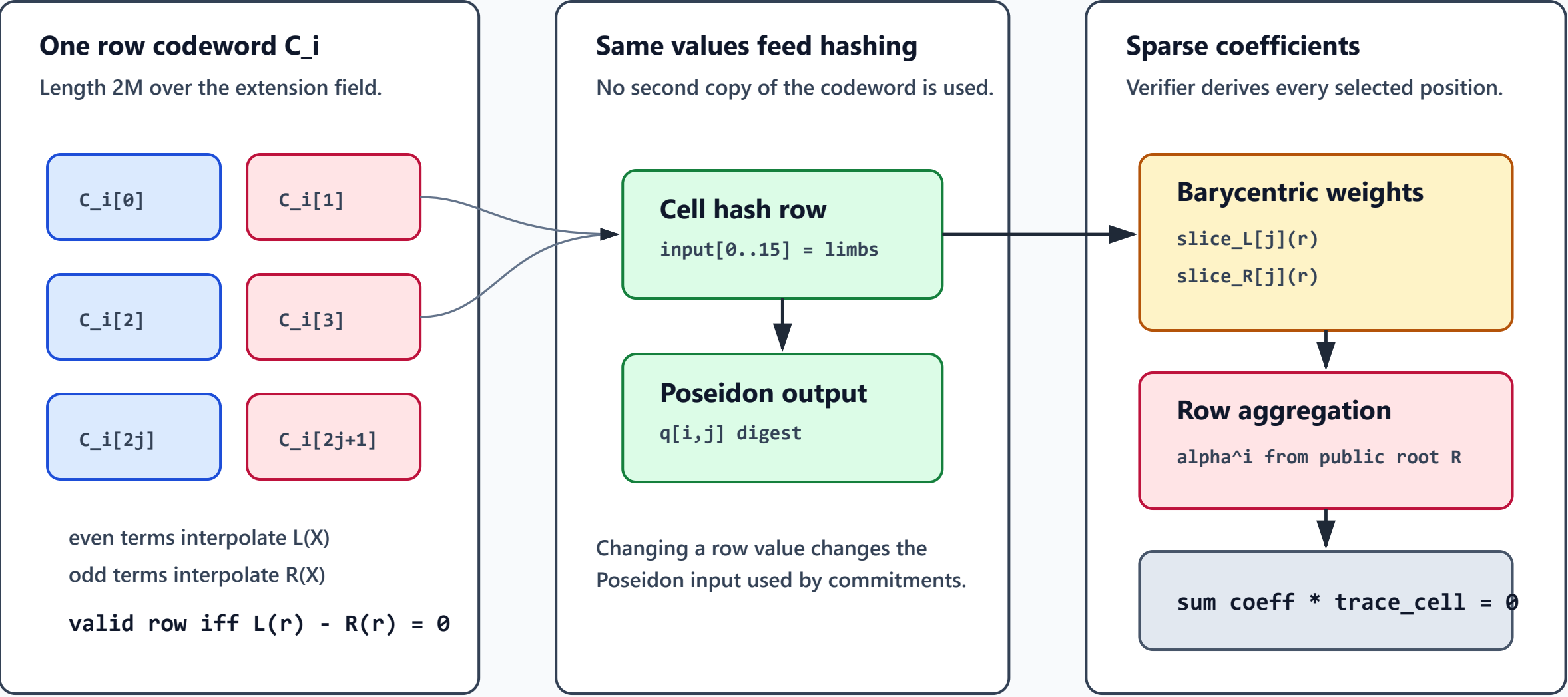
sum_i alpha^i * (
  sum_j slice_L[j](r) * C_i[2j]
  - sum_j slice_R[j](r) * C_i[2j+1]
) = 0
```

The binding detail is that this parity identity does not read a separate codeword witness. The implementation maps each codeword limb:

```
C_i[t].limb_k
```

to the exact `(trace_row, input_column)` where that limb is absorbed by the `cell` hash row. The parity check therefore reads the same values that are hashed into cell digests, then chained into row roots and column Merkle roots.

Row LDT: barycentric parity identity as one sparse linear claim



The sparse schedule and parity checks are combined as:

```
sparse_linear_claim =  
    parity_claim + link_mix * hash_link_claim
```

This gives lookup-like binding without a generic logup or VM memory table. The schedule is public, so both prover and verifier derive the linear coefficients from public shape and transcript challenges. The values being checked are WHIR-committed trace columns.

This representation does not include:

- a separate lookup table;
- memory index columns;
- table-inclusion checks for every cell and digest edge.

This is a specialized representation for a fixed public schedule rather than a general lookup system.

4. Minimal Proving Machinery

The proof has three layers.

4.1 Poseidon permutation constraints

The `LeanAirDedicatedHashAir` table constrains each row to be a valid Poseidon16 compression:

```
input[16] -> Poseidon1-16 -> output[8]
```

It stores enough intermediate round state to keep constraints low degree:

- beginning full-round post-states;
- partial-round first-lane values;
- ending full-round post-states;
- final output limbs.

The partial-round block uses the backend low-degree AIR path:

```
low_degree_air() = Some((3, DEDICATED_HASH_PARTIAL_ROUNDS))
```

This keeps the partial-round constraints at degree 3 inside the AIR sumcheck.

4.2 Public root constraints

The final row of the Poseidon schedule must output the public commitment:

```
trace[output_limb][final_row] = D[limb]
```

These are passed to WHIR as `SparseStatement::unique_value(...)` constraints.

4.3 Virtual linear schedule constraints

The schedule constraints are not local AIR transitions. They are global linear identities over the committed columns:

```
source_digest_limb - sink_input_limb = 0  
zero_padding_limb = 0  
parity_weighted_codeword_limb_sum = 0
```

They are proved by the dense and sparse `OuterSumcheckSession`s and then tied back to WHIR openings at the sumcheck point.

The split is:

- Poseidon arithmetic is local and belongs in AIR;
- deterministic schedule wiring is global and represented as a virtual linear claim;
- row low-degree testing is naturally a global linear identity.

5. Benchmark Snapshot

The latest Graviton runs give the current performance shape.

machine	vCPU	highest observed aggregate wall throughput	highest observed aggregate prove throughput
c8g.16xlarge	64	12,774 KiB/s	13,692 KiB/s
c8g.48xlarge	192	25,117 KiB/s	27,020 KiB/s

At the matching 101 row/blob payload on the 48xlarge, the available measurements cover different execution modes:

measured configuration	execution mode	throughput
LeanVM baseline parity check	single proof	1,370 KiB/s
LeanVM column-commit parity check	single proof	1,186 KiB/s
LeanAIR direct prover	single proof	~7.5 MiB/s
LeanAIR highest-throughput batch	parallel batch	25,117 KiB/s

The LeanAIR aggregate result uses parallel batching across independent proofs. Parallelized Lean-da proof generation was not measured here, so the table reports the observed configurations rather than an intrinsic speedup ratio between the two implementations. The LeanAIR single-proof value is recorded as approximate because the exact run artifact is not present in the local repo.

6. Commands

Single proof:

```
target/release/lean-air \
--log-m 13 \
--n-rows 101 \
--cell-len-ext 512 \
--prove \
--runs 5 \
--table
```

48xlarge batch shape used for highest wall throughput:

```
target/release/lean-air bench-parallel \
--log-m 13 \
--n-rows 101 \
--cell-len-ext 128 \
--concurrency 32 \
--rayon-threads 6 \
--runs-per-worker 3 \
--pin
```

LeanVM parity-check comparisons:

```
target/release/lean-da --n-blobs 101 --construction baseline
target/release/lean-da --n-blobs 101 --construction column-commit
```

Pipelined PQ blob dissemination

Status: research note **Related:** Ethereum DAS blob proof dissemination **Author:** Tau Lepton

1. Problem

Proving blobs and disseminating them at the end of the slot results in a bursty network profile and increases the bandwidth requirements for validators and builders.

Pipelining changes the shape of the same work. A node periodically broadcasts the latest proof for the covered blob set and only the new column-sample diff since the previous broadcast. Peers that already saw earlier updates do not need the old samples again.

For this note, assume two primitives:

- a proof that binds the covered blob cells to a valid data-availability statement;
- a column sample that is committed to with a commitment that is incrementally updatable, such as a Merkle tree.

The question is therefore a bandwidth-shaping question:

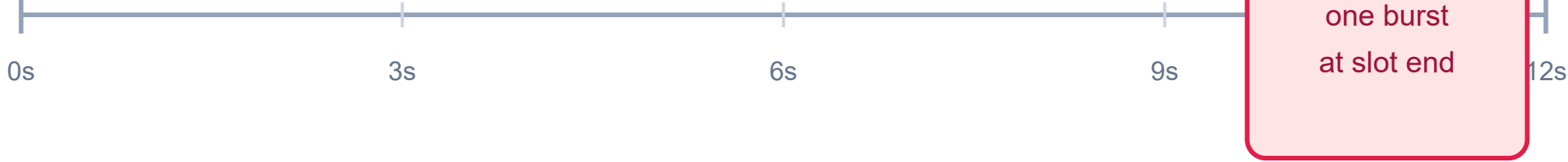
Do we prefer one low-overhead burst at the end, or repeated proof overhead in exchange for smaller bursts during the slot?

Burst profile is the issue

Same slot, different network shape: one late payload versus repeated smaller updates.

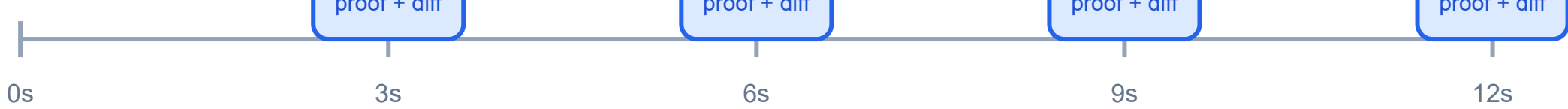
End-of-slot

send proof + all column samples near the deadline



Pipelined

send latest proof + only the new column-sample



Tradeoff

pipeline total bytes are higher because proofs repeat, but the largest per-update payload is smaller. That is useful when peak bandwidth and late verification load are the bottleneck.

2. Bandwidth Analysis

Use a concrete example, not as a parameter recommendation, but as a way to reason about the tradeoff. The analysis metric is **peak interval bandwidth**:

```
interval_bandwidth = bytes_sent_in_interval / interval_duration
peak_bandwidth     = max(interval_bandwidth over the slot)
```

This is the resource requirement a node must provision for. Total bytes still matter, but a protocol that sends fewer bytes overall can still be harder to run if those bytes all arrive in one short interval.

Assume:

```
slot_duration      = 12 seconds
update_interval    = 3 seconds
updates_per_slot   = 4
proof_bytes        = 100 kB
sampled_columns_per_update = 32
bytes_per_column_sample = 6 kB
```

Each pipelined update carries:

```
column_diff_bytes = sampled_columns_per_update * bytes_per_column_sample
                  = 32 * 6 kB
                  = 192 kB

pipelined_update_bytes = proof_bytes + column_diff_bytes
                       = 100 kB + 192 kB
                       = 292 kB
```

The full slot contains:

```
total_column_sample_bytes = updates_per_slot * column_diff_bytes
                           = 4 * 192 kB
                           = 768 kB
```

The two dissemination strategies send the same **768 kB** of column samples. They differ in when those bytes are sent and how many proofs are repeated.

End-of-slot sends one proof plus all samples in the final interval:

```
end_of_slot_total_bytes = proof_bytes + total_column_sample_bytes
                        = 100 kB + 768 kB
                        = 868 kB
```

Pipelining sends one proof plus one column-sample diff in each interval:

```
pipelined_total_bytes = updates_per_slot * pipelined_update_bytes
                      = 4 * 292 kB
                      = 1,168 kB
```

So this example pays:

```
extra_total_bytes = pipelined_total_bytes - end_of_slot_total_bytes
                  = 300 kB
```

That extra **300 kB** is exactly the three additional proof broadcasts:

```
extra_total_bytes = (updates_per_slot - 1) * proof_bytes
                  = 3 * 100 kB
```

The total-byte cost is higher for the pipeline, but the bandwidth requirement is lower because the bytes are spread across the slot.

Per-Interval Bandwidth

Divide the slot into four **3 second** intervals. The average bandwidth required inside each interval is:

A concrete bandwidth frame

Example numbers: 100 kB proof, 32 sampled columns, 6 kB per column diff, 3 second intervals.

Assumptions

slot = 12s

interval = 3s

updates = 4

proof = 100 kB

columns = 32

per-column = 6 kB

diff/update = 192 kB

End-of-slot

one proof

all sample bytes at end

final interval bytes

= 100 + 4 * 192

289.3 kB/s peak

868 kB over the final 3 seconds

Pipelined

one proof per update

only the new diff each time

interval bytes

= 100 + 192

97.3 kB/s peak

292 kB over each 3 second interval

Result: end-of-slot needs about 3x higher peak interval bandwidth than the pipelined schedule.

interval_bandwidth = interval_bytes / 3 seconds

interval	end-of-slot bytes	end-of-slot bandwidth	pipelined bytes	pipelined bandwidth
0-3s	0 kB	0 kB/s	292 kB	97.3 kB/s
3-6s	0 kB	0 kB/s	292 kB	97.3 kB/s
6-9s	0 kB	0 kB/s	292 kB	97.3 kB/s
9-12s	868 kB	289.3 kB/s	292 kB	97.3 kB/s

So the peak bandwidth requirement is:

```
end_of_slot_peak_bandwidth = 868 kB / 3 s = 289.3 kB/s
pipelined_peak_bandwidth   = 292 kB / 3 s = 97.3 kB/s
```

In this example, end-of-slot dissemination uses fewer total bytes:

```
end_of_slot_total_bytes = 868 kB
pipelined_total_bytes   = 1,168 kB
```

But end-of-slot dissemination is more resource intensive at the critical moment:

```
end_of_slot_peak_bandwidth / pipelined_peak_bandwidth
= 289.3 / 97.3
~= 3x
```

This is the main argument for pipelining. It is not a total-byte optimization; it is a peak-bandwidth optimization. It pays repeated proof bytes to avoid a large final-quarter bandwidth spike.

The rule of thumb is:

Pipelining helps when lower peak bandwidth is worth the repeated proof-byte overhead.

If blobs arrive evenly and column-sample bytes dominate proof bytes, pipelining smooths the network profile. If proof bytes dominate, or if an adversary withholds most blobs until the last interval, the smoothing benefit shrinks.